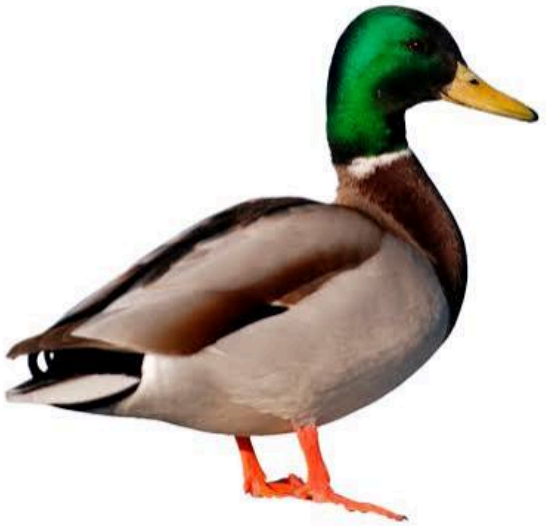
Ciência de Dados Aplicada ao Direito I

Aula 9

Funções, e a ideia de classe



Prof. Bruno da Cunha de Oliveira - 2026/2

Funções

Insper

A mesma conta em três lugares

Três prazos, e a Defensoria tem prazo em dobro. Escrevemos a conta três vezes:

```
SE parte_tem_prazo_em_dobro ENTÃO
    prazo_contestacao = 15 * 2
SENÃO
    prazo_contestacao = 15

SE parte_tem_prazo_em_dobro ENTÃO
    prazo_apelacao = 15 * 2
SENÃO
    prazo_apelacao = 15

SE parte_tem_prazo_em_dobro ENTÃO
    prazo_embargos = 5 * 2
SENÃO
    prazo_embargos = 5
```

Se a regra do dobro mudar, são três lugares para corrigir, e um deles vai ser esquecido.

Vamos dar um nome a isso

A conta é sempre a mesma: recebe um prazo em dias e uma resposta sobre o dobro, e devolve o prazo certo. Vamos escrever essa conta uma vez só, com um nome, de um jeito que dê para usá-la nos três lugares.

Não existe sintaxe certa ainda. O que importa é: o que entra, o que sai, e como se chama.

Vamos dar um nome a isso

RESPOSTA

```
FUNÇÃO prazo_final(prazo_normal, tem_prazo_em_dobro):
    SE tem_prazo_em_dobro ENTÃO
        DEVOLVER prazo_normal * 2
    SENÃO
        DEVOLVER prazo_normal

prazo_contestacao = prazo_final(15, parte_tem_prazo_em_dobro)
prazo_apelacao    = prazo_final(15, parte_tem_prazo_em_dobro)
prazo_embargos    = prazo_final(5, parte_tem_prazo_em_dobro)
```

A conta existe uma vez, com nome. Cada linha de baixo a chama com entradas diferentes e guarda o que ela devolve.

Função

Em Python: `def`

Um nome para uma sequência de instruções que recebe entradas e devolve uma saída. Escreve-se uma vez e chama-se pelo nome quantas vezes for preciso.

EXEMPLO

```
FUNÇÃO nome(entrada_1, entrada_2):
    ... o que ela faz ...
    DEVOLVER saida

resultado = nome(valor_1, valor_2)
```

Definição e chamada

```
FUNÇÃO prazo_final(prazo_normal, tem_prazo_em_dobro):    ← DEFINIÇÃO: escreve o que a função faz
    SE tem_prazo_em_dobro ENTÃO
        DEVOLVER prazo_normal * 2
    SENÃO
        DEVOLVER prazo_normal

prazo = prazo_final(15, VERDADEIRO)                       ← CHAMADA: manda a função rodar
```

A marca da chamada é o parêntese depois do nome: `prazo_final(15, VERDADEIRO)`. É ele que manda a função rodar com o que está dentro dele.

Definir não executa nada. O que executa é a chamada.

Entrada e saída

```
FUNÇÃO prazo_final(prazo_normal, tem_prazo_em_dobro):    ← ENTRADAS: o que a função recebe
    SE tem_prazo_em_dobro ENTÃO
        DEVOLVER prazo_normal * 2                        ← SAÍDA: o que volta para quem chamou
    SENÃO
        DEVOLVER prazo_normal                            ← SAÍDA, pelo outro caminho
```

A definição fixa duas coisas. As entradas, entre parênteses depois do nome, e a saída, na linha do `DEVOLVER`. O que fica entre as duas é a conta.

Uma função se descreve pelo que entra e pelo que sai.

Variável ou valor?

```
resultado = "PROCEDENTE"
```

Nesta linha, qual é a variável e qual é o valor? E qual é a diferença entre os dois?

Variável, valor e tipo

```
resultado = "PROCEDENTE"
```

Elemento	Neste caso
variável	`resultado`, o nome estável do lugar onde registramos
valor	`"PROCEDENTE"`, o conteúdo registrado neste caso
tipo	a regra sobre quais valores cabem ali

Parâmetro e argumento

```
FUNÇÃO prazo_final(prazo_normal, tem_prazo_em_dobro):
    ...

prazo = prazo_final(15, VERDADEIRO)
```

Elemento	Neste caso
parâmetro	`prazo_normal`, o nome escrito na definição, como a variável
argumento	`15`, o que entra na chamada, como o valor

A cada chamada, o parâmetro recebe o argumento. Nesta, `prazo_normal` vale 15. Em outra chamada, vale outra coisa.

Abrir o pote, agora sim

Na aula 5, “abre o pote” foi uma falha: parecia uma instrução e era um roteiro inteiro escondido num verbo.

```
FUNÇÃO abrir_o_pote(pote):
    segurar o pote com uma mão
    girar a tampa no sentido anti-horário até soltar
    tirar a tampa e apoiar na mesa
    DEVOLVER o pote aberto

pote_aberto = abrir_o_pote(pote_de_nutella)
```

Depois que a função existe e está escrita, “abrir o pote” deixa de ser falha e vira instrução legítima. O roteiro não sumiu: ganhou nome.

O que sai daqui

O que fica guardado em `p` depois desta chamada?

```
FUNÇÃO prazo_final(prazo_normal, tem_prazo_em_dobro):
    SE tem_prazo_em_dobro ENTÃO
        DEVOLVER prazo_normal * 2
    SENÃO
        DEVOLVER prazo_normal

p = prazo_final(5, FALSO)
```

O que sai daqui

RESPOSTA

5.

Na chamada, `prazo_normal` vale 5 e `tem_prazo_em_dobro` vale FALSO. O `SE` cai no `SENÃO`, e a função devolve `prazo_normal` como veio. O `DEVOLVER` é o que sai; sem ele, `p` ficaria com nada.

Esqueci o DEVOLVER

Tirei as duas linhas de `DEVOLVER` e guardei o resultado numa variável. O que fica em `p`?

```
FUNÇÃO prazo_final(prazo_normal, tem_prazo_em_dobro):
    SE tem_prazo_em_dobro ENTÃO
        prazo = prazo_normal * 2
    SENÃO
        prazo = prazo_normal

p = prazo_final(15, VERDADEIRO)
```

Esqueci o DEVOLVER

RESPOSTA

Nada. `prazo` nasceu dentro da função e fica lá.

A função calculou 30 e guardou em `prazo`, mas não entregou nada a quem chamou. `p` recebe o que a função devolve, e ela não devolveu. Quem quer usar o valor depois precisa do `DEVOLVER`.

Dentro e fora da função

DENTRO

Os parâmetros, como `prazo_normal`, e as variáveis criadas ali, como `prazo`. Existem enquanto a função roda e somem quando ela acaba.

FORA

Só o que a função devolve, e só se quem chamou guardar numa variável: `p = prazo_final(15, VERDADEIRO)`.

Só o que é devolvido existe fora da função.

Três fases, três funções

Insper

A dosimetria numa frase

Código Penal. Art. 68, caput.

> **Art. 68 -** A pena-base será fixada atendendo-se ao critério do art. 59 deste Código; em seguida serão consideradas as circunstâncias atenuantes e agravantes; por último, as causas de diminuição e de aumento.

Três fases, separadas por ponto e vírgula. Cada uma recebe o que a anterior produziu.

A saída de uma é a entrada da seguinte

```
FUNÇÃO fixar_pena_base(pena_minima, pena_maxima, circunstancias):
    ... critério do art. 59 ...
    DEVOLVER pena_base

FUNÇÃO aplicar_atenuantes_e_agravantes(pena_base, atenuantes, agravantes):
    ...
    DEVOLVER pena_intermediaria

FUNÇÃO aplicar_causas(pena_intermediaria, diminuicoes, aumentos):
    ...
    DEVOLVER pena_definitiva
```

Três funções, uma por fase. O que a primeira devolve, `pena_base`, é o primeiro parâmetro da segunda; o que a segunda devolve é o primeiro da terceira. As reticências são a regra de cada fase, que ainda vamos escrever.

Criando nossos próprios tipos

Insper

O que um texto sabe fazer

```
resultado = "PROCEDENTE"

resultado.lower()          → "procedente"
"Parcialmente".upper()    → "PARCIALMENTE"
```

Um texto já vem com coisas que sabe fazer: passar para minúsculas, para maiúsculas. Pede-se com ponto e parêntese, como uma chamada de função, só que grudada na coisa.

O que uma tabela carrega e sabe fazer

```
tabela.shape     → (200, 5)
tabela.head()    → as cinco primeiras linhas
```

No módulo 3, a tabela de dados é um tipo, o `DataFrame`. Ela carrega o próprio tamanho, 200 linhas e 5 colunas, e sabe mostrar o começo de si mesma.

`shape` se lê, sem parêntese. `head()` se chama, com parêntese.

Atributo e método

ATRIBUTO

```
tabela.shape
```

Um dado que a coisa carrega. Lê-se, sem parêntese.

MÉTODO

```
resultado.lower()
```

Algo que a coisa faz. Chama-se, com parêntese, como uma função.

Os dois se pedem com ponto. O parêntese diz qual é qual.

Podemos criar os nossos

A CLASSE É O MOLDE

```
CLASSE Cachorro:
    nome      → "Rex"
    raca      → "vira-lata"
    idade     → 3
    latir()   → "au au"
    sentar()  → o cachorro senta
```

CADA CACHORRO É UM OBJETO

```
rex = Cachorro("Rex", "vira-lata", 3)
toto = Cachorro("Totó", "poodle", 8)

rex.nome      → "Rex"
toto.nome     → "Totó"
rex.idade     → 3
toto.idade    → 8
toto.latir()  → "au au"
```

Três atributos e dois métodos, definidos uma vez no molde. Cada cachorro sai dele com os próprios valores: Rex e Totó são dois, do mesmo tipo.

Um tipo jurídico: Processo

```
CLASSE Processo:
    numero                  → "0000000-00.2026.8.26.0100"
    partes                  → [{"nome": "Ana", "papel": "AUTORA"}, ...]
    pecas                   → [{"tipo": "PETIÇÃO INICIAL", ...}, ...]
    data_sentenca           → 01/09/2026
    data_transito           → NULO
    transitou_em_julgado()  → FALSO
    juntar_peca(peca)       → acrescenta peca em pecas
    precisa_revisao()       → VERDADEIRO
```

Cinco atributos, os campos do dicionário da aula 8, e três métodos, as perguntas e ações que se fazem a um processo.

Para que criar um tipo

1. Todo processo saído do molde tem os mesmos campos. No dicionário, `data_transito` podia faltar numa peça e existir na outra.

2. A regra de decidir vive junto do dado, escrita uma vez. Quem tem um processo não refaz a conta: pergunta.

3. O código lê como a frase jurídica:

```
PARA CADA processo EM processos:
    SE processo.precisa_revisao() ENTÃO
        revisar processo
```

Função ou método

FUNÇÃO

```
contar_sem_transito(decisoes)
```

Existe solta. Recebe a lista como entrada.

MÉTODO

```
decisoes.contar_sem_transito()
```

Pertence à coisa. A lista já sabe contar a si mesma.

Um método é uma função que mora dentro de um tipo e age sobre o objeto. O ponto é a marca: `coisa.metodo()`.

Programação orientada a objetos

É o nome dessa forma de programar: o dado e o que se faz com ele andam juntos, num objeto, e o objeto sai de um tipo que nós definimos. Python é orientada a objetos: tudo nela, texto, lista, tabela, é objeto.

1. Com variáveis soltas, `numero`, `partes`, `pecas` de duzentos processos são seiscentas variáveis para não misturar. No objeto, cada processo carrega as suas.

2. Com funções soltas, quem chama precisa passar tudo: `precisa_revisao(data_sentenca, data_transito)`. O método já sabe onde está: `processo.precisa_revisao()`.

3. Mudou a regra? Muda-se num lugar, dentro do tipo, e todo processo passa a decidir do jeito novo.

Amanhã

Na aula de amanhã, quarta 09/09, vamos preparar nosso ambiente para começar a trabalhar com Python.